SUPERVISED MACHINE LEARNING

A Beginner-to-Practical Course

Understand labelled data, regression, classification, model training, evaluation, and practical Python implementation.

LEVEL	Beginner to intermediate
FOCUS	Core concepts + Python / scikit-learn
PREREQUISITES	Basic Python and elementary statistics
SUGGESTED PACE	3–4 weeks, with hands-on practice

Includes algorithms, equations, Python examples, evaluation metrics, common mistakes, exercises, and a complete beginner project.

How to use this course

Read each concept, understand the example, then implement it yourself. Use a notebook or Python script, inspect the data, and compare model performance on data the model has not seen during training.

Learning outcomes

• Explain supervised learning and distinguish it from unsupervised and reinforcement learning.

• Identify features, labels, training data, validation data, and test data.

• Understand regression and classification algorithms.

• Build machine learning pipelines using Python and scikit-learn.

• Evaluate models with appropriate metrics and interpret common failure modes.

• Recognize overfitting, underfitting, data leakage, and class imbalance.

Prerequisites and setup

You should know basic Python: variables, functions, lists, dictionaries, loops, and importing libraries. Familiarity with mean, median, probability, and graphs is useful.

```
python -m venv .venv
# Activate the environment, then install:
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

Run examples in Jupyter Notebook, Google Colab, or a Python IDE. The code uses the scikit-learn API; check the installed package documentation if an API changes.

1. What is Machine Learning?

Machine Learning (ML) is a field of computing in which algorithms learn patterns from data and use those patterns to make predictions, decisions, or useful representations. Instead of writing every rule manually, a developer provides examples and a learning procedure.

Main types of machine learning

Type	Training signal	Typical task
Supervised learning	Input examples with known target labels	Predict house price; classify spam
Unsupervised learning	Input data without target labels	Cluster customers; reduce dimensions
Semi-supervised learning	A small labelled set plus a larger unlabelled set	Image classification with limited labels
Reinforcement learning	Rewards or penalties from interaction	Learn a game-playing policy

2. What is Supervised Learning?

Supervised learning is a machine learning approach in which a model learns a mapping from input features (X) to a known target (y) using labelled examples. After training, the model predicts targets for new, unseen inputs.

Basic example

Suppose a dataset contains house area, number of bedrooms, location-related features, and the sale price. The features are the inputs X, while sale price is the target y. The model learns from historical examples and predicts a price for another house.

```
Features (X)                Target (y)
Area | Bedrooms | Age   ->    House price
1000 |    2     |  10   ->    45 lakh
1500 |    3     |   5   ->    68 lakh
 900 |    2     |  15   ->    39 lakh
```

Mathematical view

Given labelled observations D = {(x■, y■), (x■, y■), ..., (x■, y■)}, the learning algorithm estimates a function f such that f(x) produces a useful prediction of y. A loss function measures prediction error, and training seeks model parameters that reduce this loss.

A simplified objective is: minimize the average training loss over the dataset, often with a regularization term to discourage unnecessary model complexity.

Key terminology

• **Sample / observation:** one row or example in the dataset.

• **Feature:** an input variable used to make a prediction.

• **Label / target:** the known answer the model learns to predict.

• **Model:** the learned relationship between inputs and outputs.

• **Training:** estimating model parameters from labelled examples.

• **Inference:** using a trained model to predict for new data.

3. Regression and Classification

Supervised learning problems are commonly grouped by the type of target variable.

Regression

Regression predicts a numerical, usually continuous, target such as price, temperature, demand, or delivery time.

• House price prediction

• Sales or revenue forecasting

• Predicting energy consumption

• Estimating travel or delivery duration

Classification

Classification predicts a class or category. The output may be a class label or a probability for each class.

• Email: spam or not spam (binary classification)

• Disease screening: positive or negative (binary classification)

• Classifying an image as cat, dog, or bird (multiclass classification)

• Assigning multiple tags to a document (multilabel classification)

Important distinction: Logistic Regression is commonly used for classification despite having “regression” in its name. It models class probabilities.

4. The Supervised Learning Workflow

Step	What to do	Why it matters
1. Define the task	Specify target, prediction unit, and success metric	Prevents solving the wrong problem
2. Collect data	Gather representative, relevant labelled examples	Model quality depends on data quality
3. Explore data	Inspect schema, distributions, missing values, and target	Finds errors and useful patterns
4. Split data	Create training, validation, and test sets	Measures generalization honestly
5. Preprocess	Impute, encode, scale where appropriate	Makes data usable by algorithms
6. Train	Fit a baseline and candidate models	Learns patterns from training data
7. Tune	Select hyperparameters using validation or cross-validation	Improves model selection
8. Evaluate	Assess once on held-out test data	Estimates performance on unseen data

Step	What to do	Why it matters
9. Deploy and monitor	Serve predictions and monitor drift/performance	Keeps the model useful over time

Train, validation, and test sets

• **Training set:** used to fit model parameters.

• **Validation set:** used to compare models and tune hyperparameters.

• **Test set:** held aside until final evaluation; do not repeatedly tune against it.

A common starting split is 70% training, 15% validation, and 15% test, but the right split depends on dataset size and structure. For small datasets, cross-validation can use data more efficiently. For time series, split chronologically rather than randomly.

5. Data Preprocessing

Missing values

Check missingness before choosing a strategy. Common approaches include dropping rows or columns when justified, imputing numerical values with a median, or imputing categorical values with the most frequent category or an explicit unknown label.

Categorical encoding

Most traditional estimators require numeric inputs. One-hot encoding represents categories as indicator columns. Ordinal encoding is appropriate only when categories have a meaningful order or the model can handle the chosen representation.

Feature scaling

Standardization transforms a feature using its training-set mean and standard deviation: $z = (x - \mu) / \sigma$. Min-max scaling maps values to a chosen range, commonly [0, 1]. Scaling is important for distance- and gradient-based methods such as k-nearest neighbors, SVMs, and many linear models. Tree-based models usually do not require it.

Data leakage

Data leakage occurs when information unavailable at prediction time influences training or model selection. Examples include scaling the entire dataset before splitting, using future information to predict the past, or including a feature derived from the target. Fit preprocessing only on training folds; scikit-learn Pipelines help enforce this.

6. Regression Algorithms

6.1 Linear Regression

Linear Regression models a numerical target as a weighted sum of input features plus an intercept.

For one feature: ■ = b■ + b■x. For multiple features: ■ = b■ + b■x■ + ... + b■x■.

• Simple, fast baseline and relatively interpretable.

• Can underfit when the true relationship is strongly nonlinear.

• Sensitive to outliers and multicollinearity; regularized variants can help.

6.2 Decision Tree Regressor

A decision tree repeatedly splits the feature space into regions and predicts a value for each region. It can model nonlinear relationships and interactions, but deep trees can overfit.

6.3 Random Forest Regressor

A Random Forest combines many decision trees trained with randomness in samples and features. Averaging their predictions often improves stability and reduces variance compared with a single tree.

6.4 Support Vector Regression (SVR)

SVR fits a function while allowing errors within an epsilon-insensitive margin. Kernels can model nonlinear relationships. Feature scaling is usually important, and training may become costly on large datasets.

6.5 Gradient Boosting

Boosting builds an additive sequence of weak learners, commonly trees, where later learners focus on errors from earlier ones. It can achieve strong performance but needs careful tuning and validation.

7. Classification Algorithms

7.1 Logistic Regression

Logistic Regression maps a linear score to a probability using the sigmoid function: p = 1 / (1 + e■■). A decision threshold converts probabilities to class labels. It is a strong baseline and often benefits from feature scaling.

7.2 K-Nearest Neighbors (KNN)

KNN predicts using the labels of nearby training samples. It is simple but sensitive to feature scales, distance choice, and the value of k. Prediction can be slow with large training sets.

7.3 Decision Tree Classifier

A classification tree splits data to make child groups more class-pure, using criteria such as Gini impurity or entropy. Trees are easy to explain but can overfit without constraints.

7.4 Random Forest Classifier

A Random Forest combines predictions from many decision trees. It is a versatile baseline for tabular data and can model nonlinear patterns. It is less transparent than a single small tree.

7.5 Support Vector Machine (SVM)

An SVM finds a decision boundary with a large margin between classes. Kernel functions allow nonlinear boundaries. Scaling is important; probability estimates may require additional calibration.

7.6 Naive Bayes

Naive Bayes applies Bayes' theorem with a conditional-independence assumption between features given the class. It is fast and useful in some text-classification tasks, though the assumption may be unrealistic.

7.7 Gradient Boosted Trees

Gradient boosting combines trees sequentially to reduce a chosen loss. Implementations such as HistGradientBoosting, XGBoost, LightGBM, and CatBoost are widely used for structured data; external packages have separate installation and licensing details.

8. Important Evaluation Metrics

Regression metrics

• **MAE:** mean absolute error. MAE = (1/n) Σ |y■ – ■■|. It is in the target's units and is less sensitive to large errors than MSE.

• **MSE:** mean squared error. MSE = (1/n) Σ (y■ – ■■)². It penalizes large errors more strongly.

• **RMSE:** square root of MSE. It is in the target's units.

• **R²:** coefficient of determination. It compares model residual error with a mean-prediction baseline; it can be negative on test data.

Classification metrics

For binary classification, define the confusion matrix:

	Predicted positive	Predicted negative

Actual positive	True Positive (TP)	False Negative (FN)
Actual negative	False Positive (FP)	True Negative (TN)

• **Accuracy:** (TP + TN) / (TP + TN + FP + FN). Overall fraction of correct predictions; can mislead on imbalanced data.

• **Precision:** TP / (TP + FP). Of predicted positives, how many were actually positive?

• **Recall / Sensitivity:** TP / (TP + FN). Of actual positives, how many were found?

• **Specificity:** TN / (TN + FP). Of actual negatives, how many were correctly rejected?

• **F1-score:** harmonic mean of precision and recall: 2PR / (P + R).

• **ROC-AUC:** measures ranking ability across classification thresholds using the ROC curve.

• **PR-AUC / Average Precision:** summarizes the precision-recall trade-off and is often informative for rare positive classes.

Choose metrics based on the cost of errors. For fraud detection, missing a fraud case may be costly; for a high-volume alerting system, false alarms may also be expensive. Select a threshold using validation data and the real operating requirements.

9. Overfitting and Underfitting

Underfitting occurs when a model is too simple to capture the relevant structure. It performs poorly on both training and unseen data. **Overfitting** occurs when a model learns noise or idiosyncrasies in training data, achieving strong training performance but weaker validation/test performance.

• Use cross-validation to estimate variability.

• Limit model complexity (for example, tree depth or number of leaves).

• Use regularization for suitable models.

• Collect more representative data or improve feature quality.

• Use early stopping where supported.

• Keep test data isolated from model and hyperparameter decisions.

Bias-variance intuition

High bias is associated with overly simple assumptions and underfitting. High variance is associated with a model that changes substantially across training samples and may overfit. The goal is good generalization, not perfect training accuracy.

10. Class Imbalance

A dataset is imbalanced when one class is much more common than another. A model can achieve high accuracy by mostly predicting the majority class while failing to detect the minority class.

Possible approaches

• Inspect class counts and use stratified train/test splits for classification where appropriate.

• Use class_weight='balanced' for estimators that support it.

• Try over-sampling, under-sampling, or methods such as SMOTE only within training folds.

• Evaluate precision, recall, F1, PR-AUC, and the confusion matrix—not accuracy alone.

• Choose a decision threshold based on validation results and application costs.

Never apply SMOTE or any resampling operation to the full dataset before splitting. This can leak information from validation or test data into training.

11. Practical Python Example: Classification

This example uses the built-in Breast Cancer Wisconsin dataset from scikit-learn. It demonstrates a stratified split, scaling, Logistic Regression, and evaluation. The dataset is for learning and is not a clinical diagnostic tool.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, roc_auc_score
)

# Load labelled data
data = load_breast_cancer()
X, y = data.data, data.target

# Hold out test data before fitting preprocessing/model
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scaling is fitted only on training data via the pipeline
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

model.fit(X_train, y_train)

# Predict labels and probabilities
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_prob))
```

The numeric scores can vary with library versions and setup. Interpret class labels using data.target_names rather than assuming which integer represents a clinical outcome.

12. Practical Python Example: Regression

This example uses the Diabetes dataset bundled with scikit-learn. It predicts a numerical disease-progression measure for educational purposes; it is not a medical prediction system.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

data = load_diabetes()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=1.0)
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, predictions))
print("RMSE:", np.sqrt(mean_squared_error(y_test, predictions)))
print("R2:", r2_score(y_test, predictions))
```

13. Hyperparameter Tuning and Cross-Validation

A **parameter** is learned from training data (for example, linear-model coefficients). A **hyperparameter** is configured before fitting (for example, tree depth, regularization strength, or number of neighbors).

Cross-validation

In k-fold cross-validation, the training data is divided into k folds. The model is trained k times, each time using k–1 folds for fitting and the remaining fold for validation. Scores are summarized across folds.

GridSearchCV example

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

param_grid = {
    "logisticregression__C": [0.01, 0.1, 1, 10]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search = GridSearchCV(
    pipe, param_grid, cv=cv, scoring="f1",
    n_jobs=-1
)
search.fit(X_train, y_train)

print("Best parameters:", search.best_params_)
print("Best CV score:", search.best_score_)

# Final evaluation on the untouched test set
best_model = search.best_estimator_
print("Test score:", best_model.score(X_test, y_test))
```

Use a metric that matches the problem. For imbalanced classification, consider scoring='f1', 'recall', 'average_precision', or a suitable multiclass metric. For an unbiased final estimate after extensive model selection, consider nested cross-validation.

14. Common Mistakes to Avoid

• Training and evaluating on the same observations.

• Preprocessing or resampling before the train/test split.

• Using accuracy alone on a heavily imbalanced dataset.

• Ignoring duplicate rows, incorrect labels, or inconsistent units.

• Using features that would not be available at prediction time.

• Tuning repeatedly against the test set.

• Assuming a high score proves causality or guarantees real-world performance.

• Deploying without monitoring data drift, prediction quality, and operational failures.

15. Mini Project: Customer Churn Prediction

Objective: predict whether a customer will leave a subscription service (churn) using historical customer data. This is a binary classification task.

Possible input features

• Customer tenure and contract type

• Monthly charges and total charges

• Payment method and billing preferences

• Support interactions and service usage

Project workflow

• Define churn precisely and choose the prediction time horizon.

• Inspect the dataset, target balance, missing values, and duplicates.

• Split data while preventing customer-level and time-based leakage.

• Build preprocessing for numeric and categorical columns.

• Train a baseline Logistic Regression and compare a tree-based model.

• Evaluate recall, precision, F1, PR-AUC, ROC-AUC, and confusion matrix.

• Review false positives and false negatives with business stakeholders.

• Document limitations and monitor performance after deployment.

Suggested extension

Create a scikit-learn Pipeline with a ColumnTransformer: impute and scale numeric features, impute and one-hot encode categorical features, then fit a classifier. Use cross-validation for model selection and keep a final test set untouched.

16. Practice Questions

1. What is supervised learning? Explain the role of labelled data.

2. Differentiate regression and classification with two examples each.

3. What are features and target variables?

4. Why do we split data into training and test sets?

5. What is data leakage? Give one example.

6. Explain the difference between MAE, MSE, and RMSE.

7. When can accuracy be misleading?

8. Differentiate precision and recall.

9. What are overfitting and underfitting?

10. Why is feature scaling important for KNN and SVM?

11. What is cross-validation?

12. Why should SMOTE be applied only to training folds?

13. Compare Decision Tree and Random Forest.

14. What is the difference between a model parameter and a hyperparameter?

15. How would you choose an evaluation metric for a fraud-detection model?

17. Quick Revision Sheet

Concept	Remember
Supervised learning	Learn mapping from labelled inputs X to target y.
Regression	Predict a numerical target.
Classification	Predict a class or class probability.
Train set	Fit model parameters.
Validation / CV	Compare models and tune hyperparameters.
Test set	Final held-out evaluation.

Concept	Remember
Overfitting	Training performance good; unseen performance weaker.
Underfitting	Model too simple; poor fit on train and test.
Precision	TP / (TP + FP).
Recall	TP / (TP + FN).
F1	Harmonic mean of precision and recall.
MAE	Average absolute prediction error.
RMSE	Square root of average squared error.
Pipeline	Chains preprocessing and estimator safely.

18. Recommended Next Steps

• Practise Pandas and exploratory data analysis on real datasets.

• Study probability, distributions, hypothesis testing, and linear algebra basics.

• Learn feature engineering, regularization, and model interpretability.

• Explore ensemble learning and gradient boosting.

• Learn model persistence, API deployment, experiment tracking, and monitoring.

• Build end-to-end projects with reproducible preprocessing and honest evaluation.

Final reminder: a model is useful when it generalizes to the intended population and supports a well-defined decision. A high metric on one dataset is not, by itself, proof of reliability.

Documentation reference: scikit-learn User Guide and API Reference (scikit-learn.org/stable/user_guide.html). Algorithm behavior and available APIs can vary by library version.